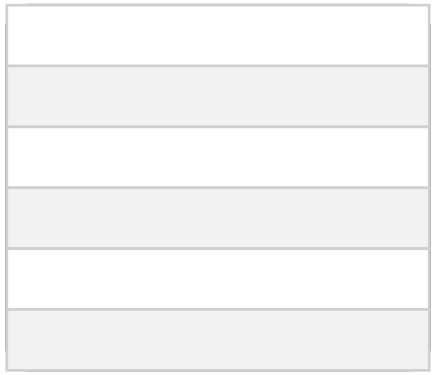
Machine Learning over Structured Data

Tutorial: relational databases, RelBench, baselines

Most real-world data lives in databases, not in one CSV

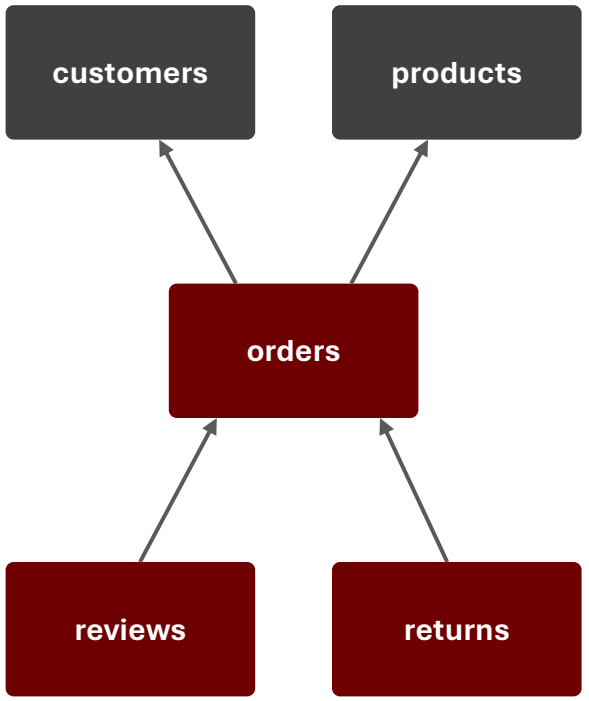
- Intro ML courses start from a single table: one row per example, one column per feature.
- In practice, data is organized in many linked tables, e.g., customers, orders, products, events…
- To predict something about an entity, the signal is spread across tables and across time.
- Today: how to set up such prediction problems correctly, and simple initial baselines.

What ML courses usually show



`data.csv`

What real data can look like



Linked through keys, and most rows carry a timestamp

Agenda and learning goals

Intro	1. Database concepts	2. RelBench	3. Prediction tasks	4. Baselines + hands-on	5. Wrap-up

By the end of today you can ...

- Read a **database schema**: tables, primary keys, foreign keys, timestamps
- Navigate **RelBench** and understand databases, tasks and evaluation measures
- Explain **temporal splits** and spot information leakage
- Implement a **simple baseline** (historical-mean) and evaluate it

A table is a DataFrame with a name

`rel-f1 · table drivers`

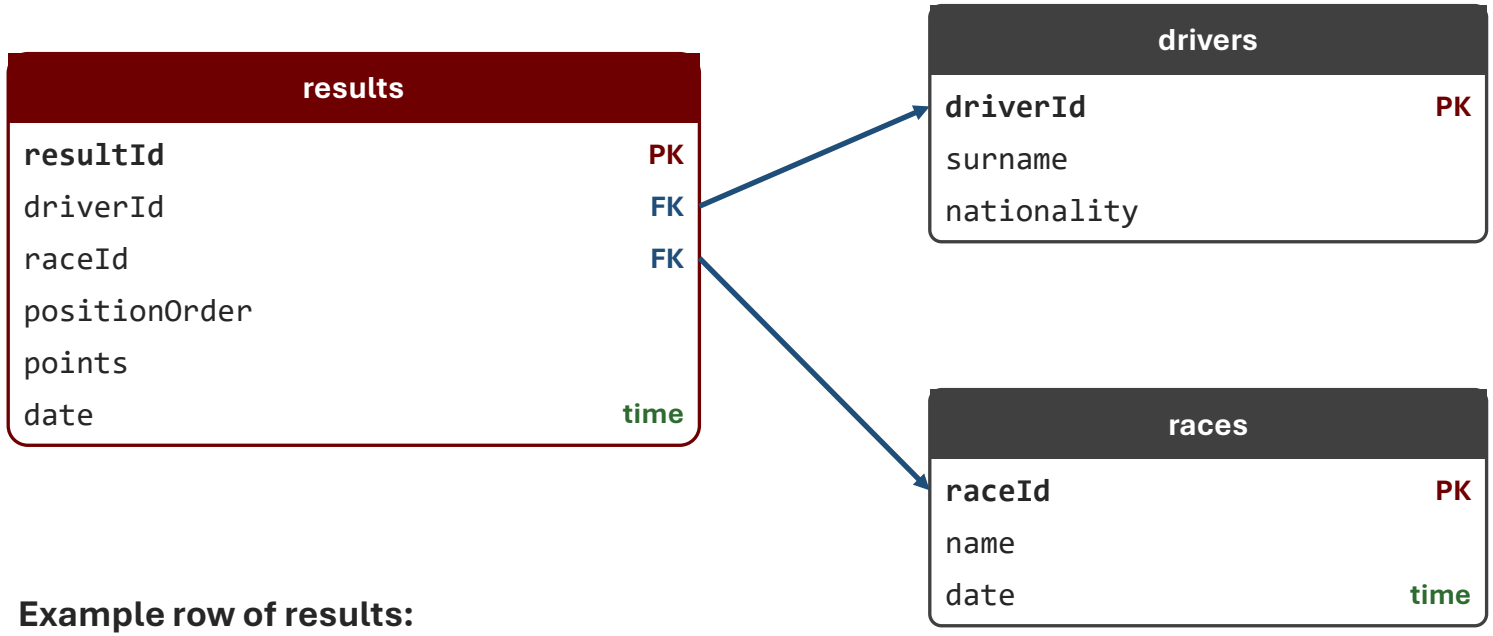
driverId	forename	surname	dob	nationality
1	Nick	Heidfeld	1977-05-10	German
4	Heikki	Kovalainen	1981-10-19	Finnish
8	Robert	Kubica	1984-12-07	Polish
30	Juan	Pablo Montoya	1975-09-20	Colombian

A = driverId, forename, surname, dob, nationality
dom(dob) = dates

each row is a tuple.

- A *tabular signature* $\mathcal{T}$ is a pair $(\mathbf{A}, \mathbf{dom})$ where:
  - $\mathbf{A} = A_1, \ldots, A_n$ is a sequence of *attribute names* (or just *attributes*)
  - $\mathbf{dom} = \mathrm{dom}(A_1), \ldots, \mathrm{dom}(A_n)$ is a sequence of *domains* (each a finite or infinite set).
- A *table* (or *relation*) r over $\mathcal{T}$ is a finite subset of $\mathrm{dom}(A_1) \times \cdots \times \mathrm{dom}(A_n)$
  - Set semantics: duplicate tuples are disallowed
- Each element t ∈ r is called a *tuple* (or *row* / *record*)

Keys link tables together



Example row of results:

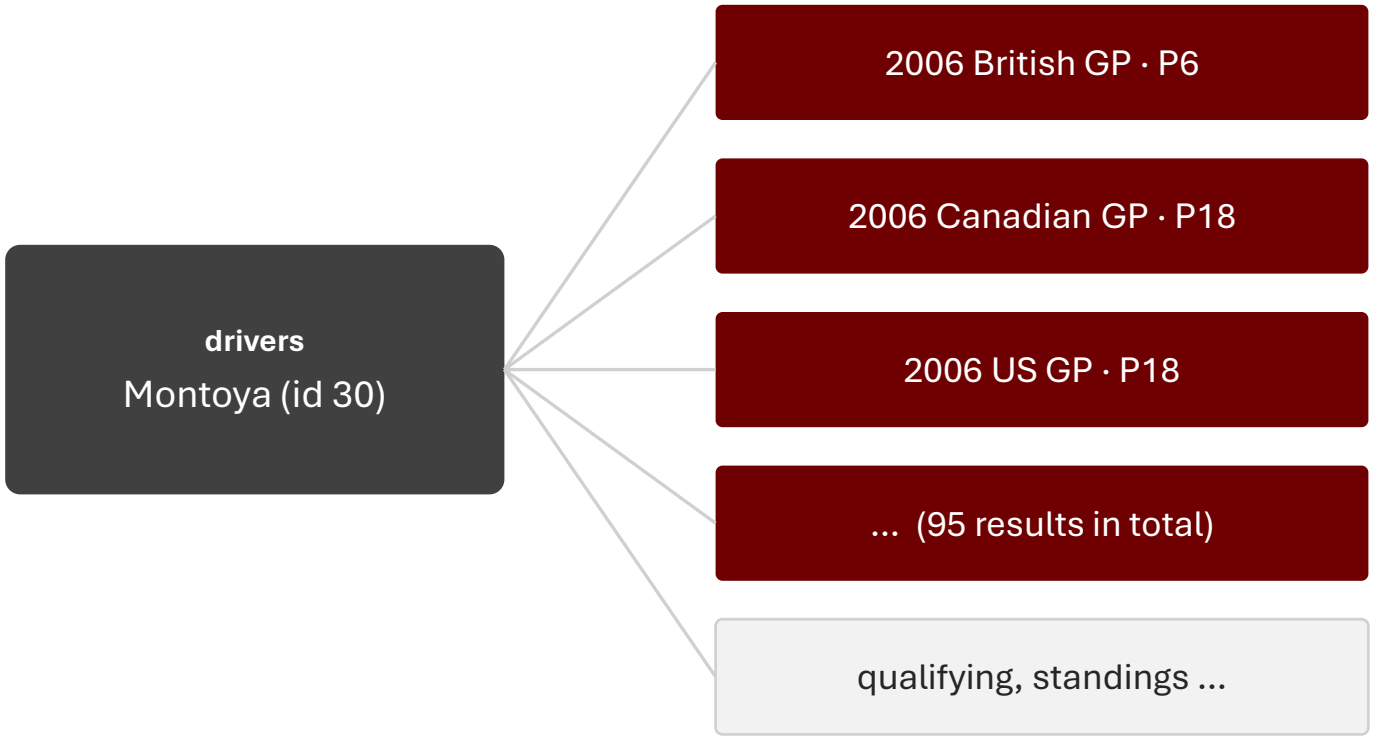
resultId	raceId	driverId	positionOrder	points	date
19008	757	30	6	3.0	2006-06-11

raceId 757 → British Grand Prix 2006 **driverId 30** → Juan Pablo Montoya

So this row says: Montoya finished 6th at Silverstone in 2006 and scored 3 points.

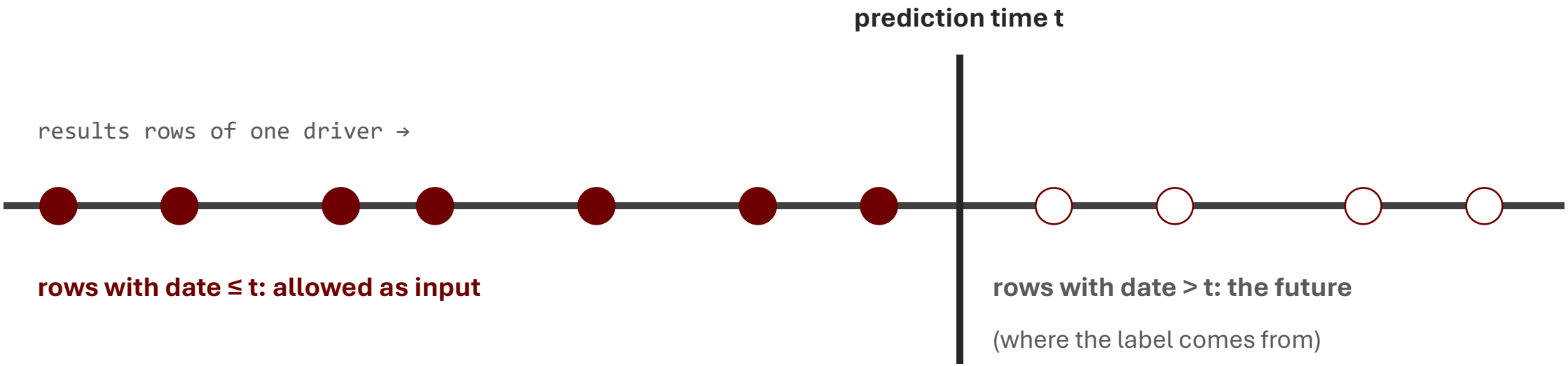
- A *primary key (PK)* is a sequence of attributes from $\mathbf{A}$
  - With distinct values for every two different tuples
  - Meaning: identify a row in its table
- A *foreign key (FK):* $R[C] \sqsubseteq \text{key}(S)$
  - Attributes in C from relation R are mapped to a key in relation S.
  - Meaning: a pointer to another table
- A join: combine two tables based on a PK-FK connection.

One-to-many: information about an entity is spread out



- One driver has many result rows (one per race)
- The driver row itself says little: name, birthday, nationality
- The predictive signal (recent finishes, crashes, points) is distributed across many-side event tables
- To get one row per driver you must **aggregate**: count, mean, last, …

Time columns and Temporal Leakage



- **Event tables** (results, races, ...) have a time column
- **Entity tables** (drivers, circuits) usually don't
- A feature used in predicting at time t may only use rows with timestamp $\leq t$
    - Otherwise the model is cheating (Temporal leakage)

RelBench: a benchmark for ML on relational databases

11
curated databases

66
tasks with fixed temporal splits

90
community databases

4
task types

- Real databases: e-commerce, sports, Q&A, clinical trials, ...
- Every task is composed of an entity, a target, a timestamp, and one standard evaluation metric
- Train / val / test split by time. Test labels kept separate to prevent accidental leakage.
- Open leaderboard, enables to submit test predictions and compare methods

```
pip install relbench

import relbench
ds = relbench.load_dataset("rel-f1")
```

Related: RelArena-α

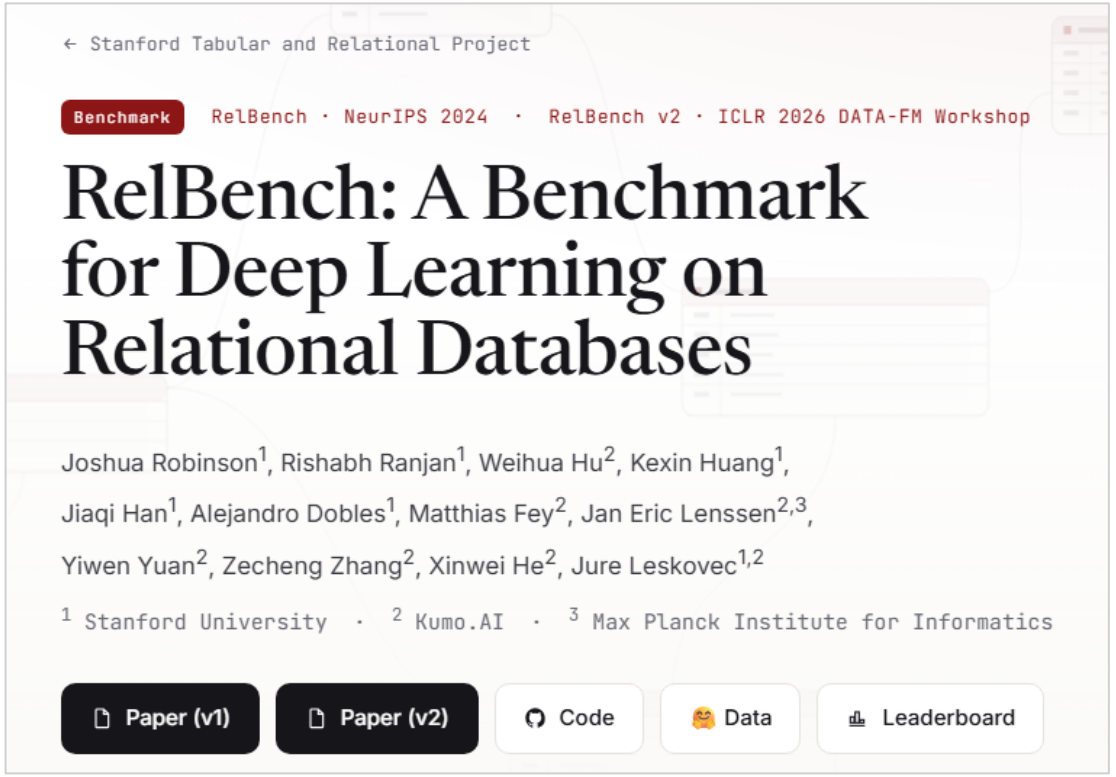
standardized train → tune → evaluate comparisons on RelBench (with Prior Labs, the makers of TabPFN)

https://github.com/PriorLabs/relarena

Demo: a tour of the RelBench website

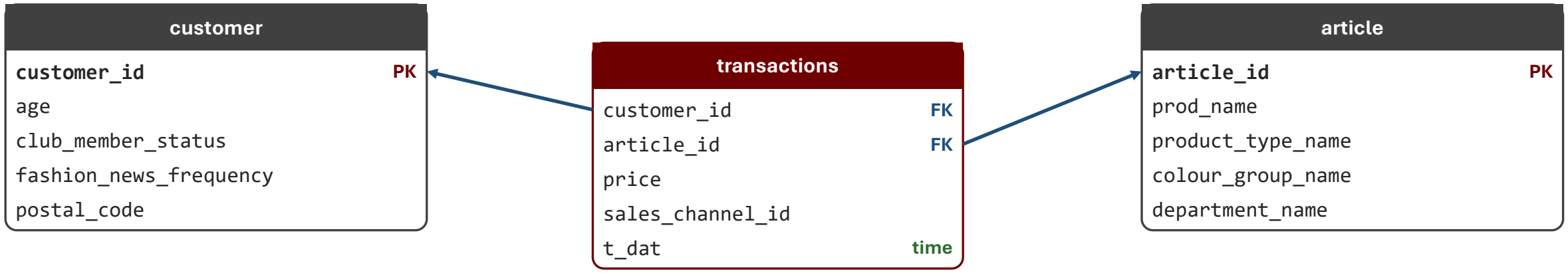
LIVE DEMO

1. **Datasets and tasks**
   the table of databases: find rel-hm and rel-f1
2. **Task types and metrics**
   AUROC, MAE, MAP; one metric per task
3. **Leaderboard**
   separate boards for classification, regression, recommendation
4. **Get started**
   a 5-line load_dataset / load_task snippet



https://star-project.stanford.edu/relbench/

rel-hm (H&M fashion): a star schema



≈1.4M customers

≈15M transactions

≈105K articles

Star schema: one central fact table of events (transactions); every other table only describes one side of an event.

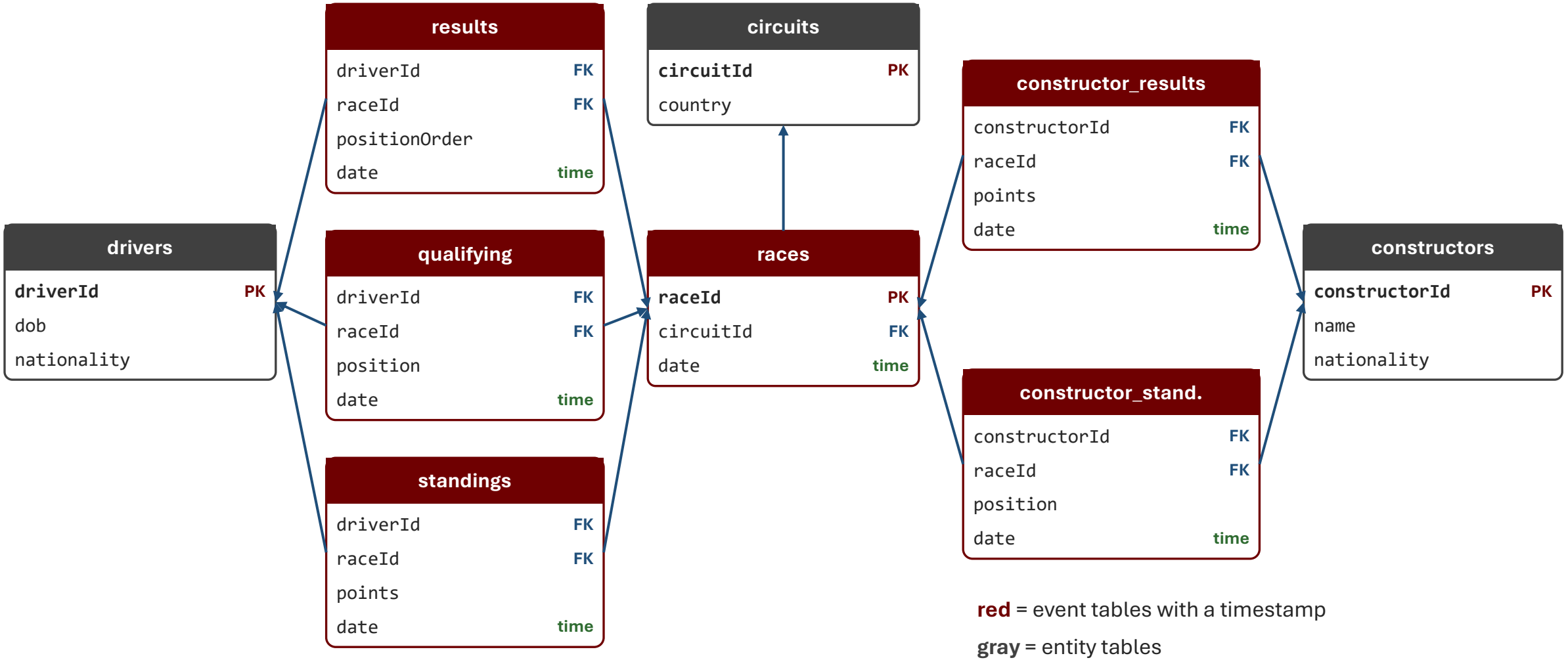
Tasks:

· user-churn (classification)

· item-sales (regression)

· user-item-purchase (recommendation)

rel-f1 (Formula 1): not a star



red = event tables with a timestamp

gray = entity tables

results and *qualifying* also point to constructors (arrows omitted)

Can the whole database be one table?

rel-hm: yes join each transaction with its customer and article

t_dat	customer	age	article	prod_name	price
09-01	c17	34	a5	Strap top	0.017
09-01	c17	34	a9	Jade jeans	0.034
09-03	c17	34	a5	Strap top	0.017
09-03	c42	51	a2	Basic tee	0.008

No information lost, only redundancy: c17's age is repeated on every purchase.

rel-f1: no natural single table

3 results rows × **3** qualifying rows = **9** joined rows

Joining two event tables of the same driver multiplies rows. Add standings, constructors … and it explodes.

To get one row per driver we must choose aggregations: mean finish? last 5 races? DNF rate?

Takeaway: a star schema is basically one big table stored compactly. Real relational DBs are not. Turning them into a table means choosing aggregations: that is **flattening** (Part 5).

Quick check: what is being predicted?

`rel-hm / user-churn`

Will a customer make no purchase in the next week?

Entity (one row per …)?

Target?

Metric?

`rel-f1 / driver-position`

Average finishing position of a driver over the next 60 days

Entity (one row per …)?

Target?

Metric?

Discuss with your neighbour for 1 minute. (Search online in RelBench if needed)

Quick check: what is being predicted?

`rel-hm / user-churn`

Will a customer make no purchase in the next week?

Entity (one row per …)? Customer

Target? churn ∈ {0,1}

Metric? AUROC

`rel-f1 / driver-position`

Average finishing position of a driver over the next 60 days

Entity (one row per …)? driver

Target? mean positionOrder over the next 60 days

Metric? MAE

Discuss with your neighbour for 1 minute. (Search online in RelBench if needed)

The Task Table

```
task.get_table("train").df
```

date	driverId	position
1950-06-19	498	18.0
1950-06-19	579	1.0
1950-06-19	589	15.0
…	…	…

`date` the prediction time t (every 60 days)

`driverId` the entity: an FK into drivers

`position` the label: mean finishing position in (t, t + 60 days]

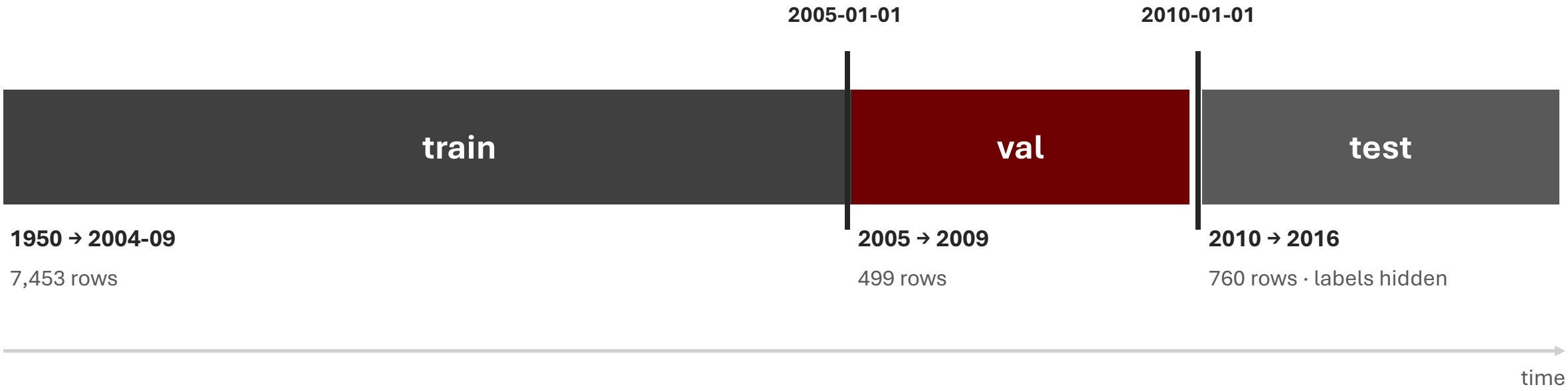
The task table is separate from the database.
Labels are computed by SQL from the future window.
Inputs come from the database up to t.
Only drivers who raced in the year before t are included.

Three task types (and their metrics)

Entity classification Yes/no about an entity	**Entity regression** A number about an entity	**Recommendation (Link prediction)** Rank items for an entity
metric: **AUROC**	metric: **MAE**	metric: **MAP@K**
• driver-dnf: will the driver fail to finish? • user-churn: will the customer stop buying?	• driver-position: average finishing position • item-sales: next-week sales of an article	• user-item-purchase: which articles will a customer buy?

The course will focus on entity prediction tasks (left 2)
Today: entity regression.
RelBench reports NMAE = MAE / std(train target), so scores are comparable across tasks.

Train/Val/Test Splits



- **Train**: fit the model. **Val**: choose models and features. **Test**: report once, at the end.
- Mimics deployment: train on the past, predict the future
- get_db() hides every row after the test timestamp, so the test window can't leak into the database you see

Temporal Leakage: using the future by accident

Random split



Training rows come from after the validation rows, so the model peeks at the future.

Temporal split



Everything the model trains on happens before what it is evaluated on.

Checkpoint question

We predict a driver's position for the 60 days after t. Can we use the driver's total career points as a feature?

Hint: total as of when?

Why start with a simple baseline?

1

Sanity check

If a fancy model can't beat "predict the average", something is wrong: a bug, leakage or bad features

2

Scale for the metric

Is MAE = 3.2 good? Only relative to a reference point

3

Surprisingly strong

On many RelBench tasks, simple per-entity statistics are hard to beat

Two baselines today

- **Global mean**: one number for everyone
- **Entity mean**: each driver's own past average

Both use only the training labels: no features, no model

Global mean vs. per-entity historical mean

PART 4 · BASELINES

Global mean

$$\hat{y} = \text{mean of all training labels}$$

Same prediction for every driver. Ignores who the driver is.

Per-driver historical mean

$$\hat{y}_d = \text{mean of driver d's training labels}$$

Uses each driver's own history. Fallback to the global mean for drivers never seen in training.

Why is this leak-free? Every training row's window ends before 2005-01-01 (the val timestamp), so all training labels are known by the time we predict on val.

Demo: loading the task in the notebook

LIVE DEMO

```
import relbench

dataset = relbench.load_dataset("rel-f1")
db = dataset.get_db()

task = dataset.load_task("driver-position")
train_df = task.get_table("train").df
val_table = task.get_table("val")
val_df = val_table.df

# a prediction for every val row, in order
pred = ...
task.evaluate(pred, val_table)   # {'nmae': ...}
```

What you should see

```
train (7453, 3) 1950-06-19 -> 2004-09-03
val   (499, 3)  2005-03-02 -> 2009-10-07
test  (760, 2)  2010-03-02 -> 2016-05-29
```

- Pass the val table explicitly; the default is test
- pred must be in the same row order as val_df
- The notebook's report() helper prints MAE and NMAE

Hands-on: implement the per-driver mean

5 MINUTES

```
# EXERCISE (notebook section 3b)
driver_means = ...   # TODO: Series indexed by driverId
pred_entity = ...    # TODO: one prediction per val row

report("entity mean", pred_entity)
```

Steps

1. Group train_df by driverId and average position
2. Map the means onto val_df["driverId"]
3. Fill drivers never seen in train with the global mean

Stretch

Use only each driver's last k training rows. Does a recent mean beat the all-time mean? Why?

Solution

```
global_mean = train_df["position"].mean()
driver_means = train_df.groupby("driverId")["position"].mean()
pred_entity = (val_df["driverId"].map(driver_means)
                                 .fillna(global_mean))
```

Baseline	val MAE	val NMAE
Global mean	[fill in]	[fill in]
Per-driver mean	[fill in]	[fill in]
Last-k mean (stretch)	[fill in]	[fill in]

- **Coverage:** share of val rows whose driver appears in train = [fill in]
- Where does the per-driver mean fail?
  - New drivers (no history), drivers changing teams, and long gaps between seasons
- This number is the bar every model in the rest of the course has to beat

Results

WRAP-UP

Model	val MAE	val NMAE	Uses entity history?	Uses other tables?
Global mean	4.334	0.617	No	No
Per-driver mean	3.714	0.529	Yes (labels)	No
Per-driver mean of last k entries	4.034	0.574	Yes (labels)	No

Discuss

What information could other tables add that can help the prediction?

Takeaways

WRAP-UP

1

Schema first

Find the entity tables, the event tables, and the keys between them

2

Time is part of the data

Split by time, and build every feature from rows before t

3

Baseline before model

The per-entity historical mean is cheap, honest and often strong

Next in the course

Deep feature synthesis – both manual and automatic.

Resources

`relbench.stanford.edu`

`structml_tutorial.ipynb`